

SPD: Slot Profile Diagrams

DEFINITIONS & PROOFS / ENGLISH EDITION

Definition, fixed version v0.1, 2026-09-14. NER expander · Python definition · standard-count decoder · well-ordering paper.

An SPD expression is a finite column diagram with four natural numbers per relation. Heads and ordinary arguments are read recursively from the referenced columns; no separate tree, level, ordinal rank, or operation history is stored. The shared DAG is an index reconstructed for comparison, not a fifth notation field.

This page fixes the highest visible head fiber, LATENT lower packages, strict connector guard, and same-head carry in later blocks. Other experimental replacement or merging rules are not SPD v0.1. Software budgets are not mathematical rules either.

The current well-ordering result is a paper argument; there is no SPD Lean certificate. Well-founded expansion on all legal diagrams and well-ordering of the column order on the standard reachable domain are different conclusions. No comparison of the **whole order types** of SPD and ARD, ARD2, wY, or IPD has been proved.

1. Finite column syntax

A finite diagram is $G = (C_0, \dots, C_{m-1})$. Column j is a finite set of relations, each of the form

$$(h, p, s, q), \quad 0 \leq h, s \leq p < j, \quad q \in \{0, \dots, p\} \cup \{j, j+1\}.$$

Here h is the head-source column, p the parent column, and s the ordinary-argument source; the child is the containing column j . The root state q is:

State	Integer encoding	List display
ROOT	$0 \leq q \leq p$	The number q
SELF	$q = j$	*
LATENT, awaiting activation	$q = j + 1$	+

LATENT is not a reference to the next column; appending column $j + 1$ later does not change its meaning. For each (h, p, s) retain only the greatest q , and ignore duplicates. Smaller roots are not simultaneously read into the head as additional visible rows.

The lossless list uses one pair of brackets per column, semicolons between relations, and commas between the four coordinates. For example:

```
[][0,0,0,*][1,1,1,*]
```

An empty column is $[]$; the zero diagram, with no columns, is $\emptyset$. A natural number m denotes m empty columns, not a complex column whose count happens to be m . Input and construction normalize by the rules above.

2. Recursive heads read from column relations

2.1 The auxiliary term order

Auxiliary finite terms are built from Z , address atoms $\text{atom}(r)$, two formal letters A, B , and

$$N(h; t_0, \dots, t_{k-1}).$$

The head h and every ordinary argument t_i belong to the same term syntax, with $k \geq 0$. The formal constant order is

$$Z < \text{atom}(0) < \text{atom}(1) < \dots < A < B,$$

and all constants are below all N terms. Compare two N terms a, b in this order:

1. If some immediate **ordinary argument** of a is at least b , then $a > b$.
2. Otherwise, if some immediate ordinary argument of b is at least a , then $a < b$.
3. Otherwise compare heads, then the numbers of arguments, then the equal-length argument lists lexicographically.

This is the specified lexicographic path order, LPO, with **head, arity** precedence. The head itself is not inserted into the ordinary-argument lists in the first two clauses. Recursion enters smaller finite terms and does not run fundamental sequences; arity precedence must not be omitted. There is no global CAP above every compound term.

The letter A denotes a future parent and B a future endpoint. Substituting $A = p$ means replacing every occurrence of A , at any depth, by $\text{atom}(p)$. The letter B remains the common greatest atom, not a top above all compound terms.

2.2 The head H_j of column j

Compute heads from left to right. Ignore LATENT rows first; if no visible row remains, put $H_j = Z$. Otherwise select the unique head index h_* with greatest formal key (H_h, h) among the visible rows. Read only the rows with $h = h_*$, in decreasing numerical address order (p, s, q) . Each contributes the argument pair

$$(H_s, Q), \quad Q = \begin{cases} A, & q = p, \\ B, & q = j, \\ \text{atom}(q), & q < p. \end{cases}$$

Concatenate these pairs and set $H_j = N(H_{h_*}; H_{s_1}, Q_1, \dots, H_{s_r}, Q_r)$, where $r \geq 1$. Visible rows with lower heads and all LATENT rows still participate in column comparison and expansion; they simply do not belong to the current head's highest visible fiber.

When H_h, H_s are referenced, their internal A, B remain formal variables rather than being instantiated at historical parent or child addresses. Every fixed ROOT is strictly before j . When H_j is later used at a parent $p \geq j$, its formal ROOT letters are below $A = p < B$, so the highest-head-fiber choice is unchanged by a legal strictly increasing address relabelling.

An implementation can share identical subexpressions and compare them with an explicit stack, without expanding repeated subtrees. The derived index is not saved with the input as frozen head data.

The proof uses the derived depth: constants have depth 0, and $d(N(h; t_0, \dots, t_{k-1})) = \max(d(h) + 1, d(t_0), \dots, d(t_{k-1}))$. The depth-initial-segment lemma gives $d(a) < d(b) \Rightarrow a < b$, allowing Python/JavaScript to distinguish terms by cached depth first. This is an equivalent shortcut for the LPO above, not a new comparison rule or another input level.

3. Direct column comparison

The profile of a relation $e = (h, p, s, q)$ in column j is

$$P(e) = (H_h[A = p], h, H_s[A = p], s, q).$$

The letter B is the same common greatest endpoint atom in all compared profiles. Compare q by its integer encoding, equivalently ROOT order followed by SELF and LATENT.

Sort each column in decreasing order of $(p, P(e))$ and compare relations lexicographically by that key, with a proper prefix smaller. Compare diagrams left to right by columns, again making a proper prefix smaller. Before the first differing column the prefixes agree, so all referenced heads are already determined in common.

Comparison does not search expansion paths.

The controller uses a different priority: maximize $(P(e), p)$ in the last column. Profiles with different parents must instantiate A at their respective parents; formal heads cannot simply be compared in one universally shared parent context.

4. The complete finite fundamental-sequence rule

The index n is a nonnegative integer, and $0[n] = 0$. For a nonempty finite diagram put $x = m - 1$ and let ∂G delete the last column. Define

$$G[0] = \partial G, \quad G[n] = \partial G \quad \text{if } C_x = \emptyset.$$

Otherwise $n > 0$ and the last column is nonempty. Fix the old controller $e = (h, c, s, q)$ and put $L = x - c > 0$. Initially retain $C_0, \dots, C_{x-1}$. Then append n blocks in order. Each consists of a lowered seam, a connector only when nonempty, and an independent pure source copy.

4.1 Relocation and block addresses

Let d_b be the accumulated displacement before block b , with $d_0 = 0$, and put

$$\phi_b(i) = \begin{cases} i, & i < c, \\ i + d_b, & i \geq c, \end{cases} \quad S_b = x + d_b, \quad c_b = \phi_b(c).$$

When a relation from column j moves to child j' , relocate its first three coordinates by the address map, but decode its root by its original role:

$$q' = \begin{cases} j' + 1, & q = j + 1, \\ j', & q = j, \\ \phi(q), & q \leq p. \end{cases}$$

Thus a general increasing map sends LATENT to $\phi(j) + 1$, not $\phi(j + 1)$. Relocate first, then lower at the destination addresses.

4.2 The lowered seam S_b

Relocate all non-controller rows of the old last column. Write the moved controller as (h_b, c_b, s_b, q_b) and lower only its root by one state:

$$\text{LATENT} \longrightarrow \text{SELF} \longrightarrow c_b \longrightarrow c_b - 1 \longrightarrow \dots \longrightarrow 0 \longrightarrow \text{delete the row.}$$

Next, for every $0 \leq k, t \leq c_b$ satisfying

$$(H_k[A = c_b], k, H_t[A = c_b], t) < (H_{h_b}[A = c_b], h_b, H_{s_b}[A = c_b], s_b),$$

insert $(k, c_b, t, \text{LATENT}_{S_b})$.

Only when $b \geq 1$, also carry from the cut column C_c of the **original input** every LATENT row and every visible row whose head index equals the original controller's h . Do not carry other visible rows. Relocate by ϕ_b and rebind the source column's own SELF/LATENT to the corresponding marker at S_b . The first block carries no source column. Normalize the union and append S_b .

Same-head carry retains the old argument fiber that can be continued. A change of head does not carry the old visible fiber, while all awaiting slots remain available.

4.3 The strict connector guard

After appending the seam, evaluate the old referenced templates in

$$\hat{P}_{S_b} = (H_{h_b}[A = S_b], \uparrow h_b, H_{s_b}[A = S_b], \uparrow s_b), \quad \uparrow i = \begin{cases} S_b, & i = c_b, \\ i, & i \neq c_b. \end{cases}$$

Only the address coordinates are raised; **do not replace the old templates by H_{S_b}** . For example, if $h_b = c_b$, the first entry is still the old cut column's head evaluated at $A = S_b$; only the second entry becomes S_b .

Enumerate $0 \leq k, t \leq S_b$. Insert $(k, S_b, t, \text{LATENT})$ in the connector if and only if

$$(H_k[A = S_b], k, H_t[A = S_b], t) < \hat{P}_{S_b}.$$

The connector is at $S_b + 1$, so its LATENT integer is $S_b + 2$. Append it only if its relation set is nonempty, setting $\varepsilon_b = 1$ in that case and $\varepsilon_b = 0$ otherwise.

The inequality is strict: equal four-coordinate keys do not pass. Either k or t may equal the new seam S_b . Connectors can therefore genuinely use newly created columns, not just a fixed old head library.

4.4 The independent pure source

Update

$$d_{b+1} = d_b + L + 1 + \varepsilon_b.$$

Copy the original $C_c, \dots, C_{x-1}$ in full under ϕ_{b+1} , placing original column j at $\phi_{b+1}(j)$. This copy is not filtered by same-head carry and is not merged into the seam. Rebind SELF/LATENT according to each copied column's own role.

The next block thus recovers exact relocation of the old source heads. One expansion uses only these finite loops and finite head comparisons. It neither recursively expands the newly produced diagram nor defines a fundamental sequence by first running a diagram to zero.

4.5 Immediate properties and an example

Each block has width $L + 1 + \varepsilon_b$ and depends only on the prefix already built, not on the final total index n . Hence $G[n]$ is a complete-column prefix of $G[n + 1]$, proper when the old last column is nonempty. All columns before the old last column remain unchanged, and the first difference is the strict lowering at the first seam. Every fundamental-sequence term of a finite nonzero diagram is therefore strictly smaller; $0[n] = 0$ is not a strict descent edge.

```
S1    = [][0,0,0,*]
S1[1] = [][0,0,0,0][0,1,1,+;0,1,0,+][]
counts: 1,3 -> 1,2,9,1
```

5. Seeds, the standard domain, and the well-ordering scope

The seed S_n has $n + 1$ columns:

$$C_0 = \emptyset, \quad C_j = \{(j-1, j-1, j-1, j)\} \quad (1 \leq j \leq n).$$

Adjoin a greatest external **Top**, displayed as **Limit** of SPD, with $\text{Top}[n] = S_n$. In particular $S_0 = []$, not zero; $S_{n+1}[0] = S_n$ and $S_0[0] = 0$.

Let $D(H)$ contain H and all its finite expansion descendants, ignoring the zero self-loop. The finite standard domain is $\text{Std} = \bigcup_{n < \omega} D(S_n)$; the complete notation additionally contains the top. Standardness means finite reachability from a seed, not the existence of an auxiliary semantic representation. Structural parsing alone does not certify handwritten lists as standard; section 7 supplies a separate membership procedure.

The well-ordering paper has the paper axiom bound $KP_\omega +$ “there exists an uncountable ordinal”, with full Set Induction and without requiring Power Set or Choice. It first ranks the auxiliary heads as an actual set order, adds strictly lower new-parent requests to the finite-demand relation, proves supply using closure under countably many witness operations, and finally lowers the represented last endpoint block by block. Neither the target well-order nor a supplier is assumed.

The conclusions are distinct:

- The nonzero expansion relation on all structurally legal finite diagrams is well-founded.
- The specified first-difference column order is a well-order on Std , also after adjoining the greatest top.

The second conclusion uses prefix-related fundamental sequences: a smaller-index term is reached from a larger-index term by repeatedly deleting last columns. Induction on the descending rank makes each seed’s descendant cone comparable by reachability. Seeds are nested prefixes, so any two standard terms have a common seed ancestor. Thus standard column order agrees with proper descendanthood.

Column order on all legal raw diagrams is not a well-order. Let R_j have j empty columns followed by the single relation $(0, j-1, 0, 0)$, for $j \geq 1$. Then $R_j > R_{j+1} > \dots$. This is a column-order descending chain, not an expansion chain. The standard-domain restriction cannot be omitted.

This is a paper proof status, not a Lean formalization of SPD or a proof about Python/JavaScript virtual machines. Local head libraries can be regenerated and reattached in actual expansions, but expressive local profiles do not automatically determine the whole diagram order. No order-type inequality between SPD and ARD, ARD2, wY, or IPD is claimed, nor is the axiom bound claimed optimal.

6. Exact local counts

Freeze the prefix before a column and treat that column as last. Make a positive-index expansion and delete the appended tail, retaining only the first seam. Repeat until that column is empty, then delete the empty column. The local count includes this final deletion, so an empty column has count 1. The first block has no source carry, and the retained local step is independent of the positive index.

For every parent p actually present in position j , order $i = 0, \dots, p$ by $(H_i[A = p], i)$, and let $r_p(i)$ be its zero-based rank. Take that parent’s highest-profile row (h, p, s, q) and define

$$\delta_p(q) = \begin{cases} q + 1, & q \leq p, \\ p + 2, & q = j, \\ p + 3, & q = j + 1. \end{cases}$$

The exact formula is

$$\text{count}(C_j) = 1 + \sum_p [\delta_p(q) + (p+3)((p+1)r_p(h) + r_p(s))].$$

Each parent independently decrements its root digit and fills lower pairs with LATENT, so every local step decreases the total by exactly one. The previous contents of lower pairs do not affect this remaining count. The program evaluates the formula directly instead of clearing the column step by step to count it.

Every legal column at position j satisfies

$$\text{count}(C_j) \leq B(j) = 1 + \sum_{p < j} (p+3)(p+1)^2 = 1 + \left(\frac{j(j+1)}{2} \right)^2 + \frac{j(j+1)(2j+1)}{3}.$$

This is a quartic bound in column position, not a bound on the complete path to zero or the number of columns in one expansion. Seed counts begin 1, 3, 16, 45, 96, 175, $\dots$; seed column $j \geq 1$ has count $j^2(j+2)$.

Equal-looking count strings in other notations are not thereby identified.

7. Finite inversion of standard counts

Every actual expansion strictly decreases the count word lexicographically: the first changed count decreases by one at a positive index, and deletion gives a proper prefix. Standard descendant cones agree with column order, so counts are strictly order-preserving and injective on Std. Raw diagrams can still have colliding count words.

7.1 Two finite-width facts

Fix a width bound $m > 0$. The seed S_{m-1} is the greatest standard diagram of width at most m . It is the first m columns of every wider seed; a diagram of width at most m lying below that wider seed cannot first exceed this prefix.

For nonempty standard G of width at most m , put

$$T_m(G) = \text{prefix}_m(G[m]).$$

This is the greatest proper standard descendant of width at most m . Taking a prefix is finitely many $[0]$ steps. When the last column is nonempty, $G[m]$ already has at least m columns and prefix-related fundamental sequences compare every other immediate term with it. Every proper descendant of width at most m lies below some immediate term, hence below these first m columns. If the last column is empty, T_m simply deletes it.

7.2 Algorithm and correctness

The input is a positive-integer word $(t_0, \dots, t_{m-1})$; the empty word returns zero. Otherwise start at $G = S_{m-1}$ and process $j = 0, \dots, m-1$:

1. Reject if column j is missing or its current count $c < t_j$. If $c = t_j$, continue to the next position.
2. If $c > t_j$, truncate to $j+1$ columns and perform $c - t_j - 1$ frozen-prefix local steps, reducing this count to $t_j + 1$.
3. Set $G := T_m(G)$. This count decreases once more to t_j , without changing the already matched earlier counts.
4. After all positions have been processed, return the normalized column diagram G .

If a target standard diagram A exists, the algorithm maintains $A \leq G$. Truncation and local lowering stay strictly above the target at the first differing position; selecting the greatest finite-width proper descendant also cannot jump below A . Valid words therefore are not rejected. Conversely, successful output is an actual standard descendant with exactly the requested counts and is unique by injectivity. A missing column or a smaller count thus proves invalidity, rather than merely reporting an unsuccessful path search.

There are at most m correction phases and at most $\sum_{j < m} B(j) = O(m^5)$ local steps, plus at most m calls to T_m . This is not an unanalyzed total time or memory complexity claim. The implementation may replace $[m]$ by any smaller positive index sufficient to reach m columns. With current last position j and control parent c , a block has at least $j - c + 1$ columns, so $\max(1, \lceil (m - j) / (j - c + 1) \rceil)$ suffices and yields the same first m columns. NER accepts $1, 3, 16$ or $C(1, 3, 16)$, both decoding to S_2 ; $C()$ is zero and $C(1, 1, 1)$ is three empty columns. A single natural number still counts empty columns. $C(1, 4)$ is invalid even though a raw two-column diagram with counts $(1, 4)$ exists.

The Python decoder provides `decode_counts`. `InvalidCounts` means that no corresponding standard term exists or that the input is not a positive-integer word. `TimeoutError` and `OverflowError` only report budget exhaustion, not invalidity, a false negative, or mathematical nontermination. Defaults allow width 32 and 100000 local steps. The whole call shares one `spd.Budget`; internal calls do not repeatedly reset its timer.

Standardness of a general raw diagram is also decidable: normalize it, compute its counts, decode the unique standard representative, and compare the complete column diagrams. **Decodable counts alone do not suffice.** The function `is_standard` implements this test. Illegal structure still raises `ValueError`; resource exceptions are not converted to `False`.

8. Software use and resource guards

8.1 NER

Import the complete `SPD.ne-rewritten.js` into NER's custom-notation facility and select **SPD**. It has no external dependency or network request. Its registration ID is `spd-research-20260914`. The default display is 列表 (list), and equivalent displays add only 计数序列 (count sequence), without a duplicate list button.

Inputs include `S0`, `S3[2]`, `Top[3][2]`, `Limit of SPD`, natural numbers, complete column lists, and the standard-count forms above. Whitespace is ignored. After evaluating an expansion path, the stored term is the actual normalized diagram, not that path. `FS`, `FS_alter`, and `FS_short` use exactly the same rule with no index offset. The top is a limit; `is_limit` classifies a finite nonzero term as a limit exactly when its last column is nonempty, and a term ending in an empty column as a successor.

Lists and counts inherit the page font. Counts use `BigInt`. Default general evaluation has an approximately 900 ms budget, with a separate approximately 200 ms count-display budget. Work, columns, relations, shared nodes, argument slots, comparisons, and parent-context caches are also bounded. The principal current geometry limits are 2048 columns, 60000 relations, and 80000 shared nodes. All constants are in the source's `LIMITS`.

Structural exhaustion raises `SPDLimit` and never returns a truncated fundamental-sequence term. An unfinished count display says “计数预算内未算完” (“counts not completed within budget”), not an approximate count. Mathematically invalid counts and execution limits are reported separately.

8.2 Python modules and direct execution

Place the files from this directory on the module search path and use Python 3.10+:

```
import spd
from spd_count_decode import decode_counts, is_standard

a = spd.seed(2)          # S2, three columns
assert spd.counts(a) == (1, 3, 16)
assert spd.fs(a, 0) == a[:-1]
assert spd.compare(spd.fs(a, 1), a) == -1
assert spd.fs(a, 1) == spd.fs(a, 2)[:len(spd.fs(a, 1))]
assert decode_counts((1, 3, 16)) == a
assert is_standard(a)
```

A diagram is a list/tuple of columns and quadruples, with normalized results returned as tuples; zero is (). The names `seed`, `normalize`, `fs`, `compare`, and `counts` are functions, not methods of an SPD class. `compare` returns `-1, 0, 1`; `read` builds a temporary derived index. The main module handles finite diagrams only and has no top object or NER text parser.

Graph operations first validate and normalize input. Negative values, booleans, and noninteger coordinates or indices are rejected. The default Budget allows 15 seconds and 2000000 events, with additional limits on nodes, argument slots, comparison pairs, parent contexts, columns, and cumulative relation allocations. The same budget may be passed to several calls. A limit is disabled only by explicitly setting it to `None`. Default guards are not permission for unbounded computation; exhaustion raises and returns no partial result.

From the repository root, run:

```
python -B notations/SPD/spd.py
python -B notations/SPD/spd_count_decode.py
```

These entry points currently run only the demonstrations at the ends of the source files. They print seed counts and an expansion, or the decoding of `(1, 3, 16)`, respectively. There is no command-line argument interface for parsing expressions or supplying expansion indices; use the function API above for programming. Independent implementations and finite tests check the rule but do not replace the well-ordering paper or constitute SPD Lean validation.